

***MICROPROCESSOR
AND
MICROCONTROLLER LAB
MANUAL***



**Electronics and Instrumentation Department
National Institute of Technology, Silchar
Assam-788010**

LIST OF EXPERIMENT

8085 MICROPROCESSOR PROGRAMMING:

Experiment No: 01a	8-Bit Addition Considering With & Without Carry.
Experiment No: 01b	8-Bit Subtraction Considering With & Without Carry.
Experiment No: 02a	Addition Of Two 16 Bit Number With & Without Carry.
Experiment No: 02b	Subtraction Of Two 16 Bit Number With & Without Carry.
Experiment No: 03	Multiplication Of Two 16 Bit Numbers.
Experiment No: 04	Division Of Two 8- Bit Number
Experiment No: 05a	Smallest Among 'N' Numbers
Experiment No: 05b	Find The Largest Number Among 'N' Numbers
Experiment No: 06a	Ascending Order Of 'N' Numbers
Experiment No: 06b	Descending Order Of 'N' Numbers
Experiment No: 07a	Hexadecimal To Decimal Code Conversions
Experiment No: 07b	Hexadecimal To Binary Code Conversions
Experiment No: 08	Fibonacci Series
Experiment No: 09	Factorial Of 'N' Data's
Experiment No: 10	Sums Of Series Of 'N' Number

8086 MICROPROCESSOR PROGRAMMING:

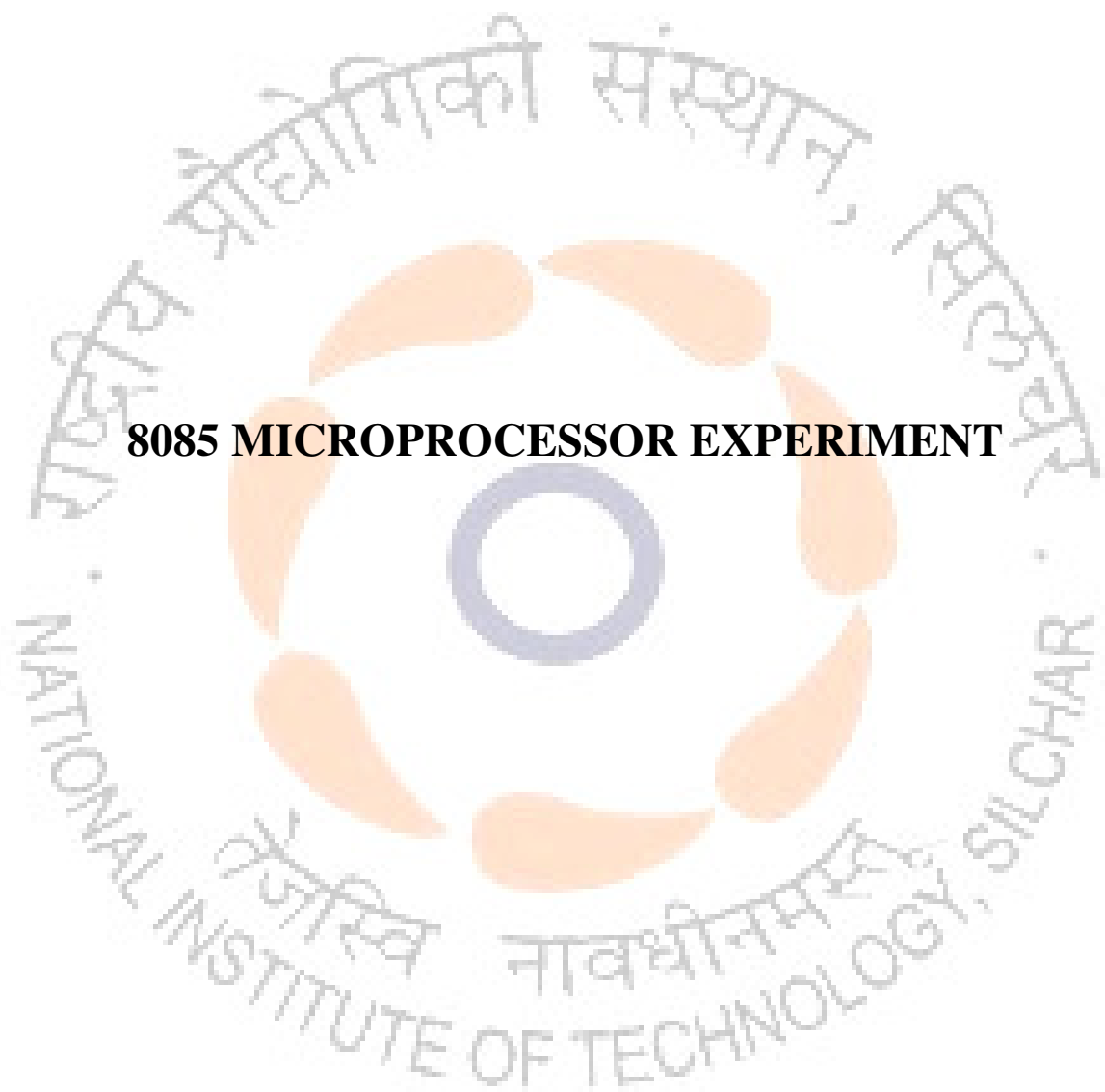
Experiment No: 11a	Addition of Two 32 Bit Number
Experiment No: 11b	Subtraction Of Two 32 Bit Number
Experiment No: 12a	16 Bit Multiplications
Experiment No: 12b	16 Bit Divisions
Experiment No: 13	Factorial of 'N' Number
Experiment No: 14	Sorting In Ascending Order
Experiment No: 15	Sum Of N Numbers In An Array

8051 MICROCONTROLLER PROGRAMMING:

Experiment No: 16	16 Bit Addition
Experiment No: 17	16 Bit Subtraction
Experiment No: 18	16 Bit Multiplication
Experiment No: 19	16 Bit Division

INTERFACING PROGRAMS:

Experiment No: 20	Stepper Motor Interfacing Using 8085
Experiment No: 21	Interfacing With Programmable Input Output – 8255



8085 MICROPROCESSOR EXPERIMENT

E & I DEPT.

Ex. No: 01a 8-BIT ADDITION CONCERNING WITH & WITHOUT CARRY.**AIM:**

To perform addition of two 8-bit numbers using 8085 microprocessor.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- Power Supply

8-BIT ADDITION:**ALGORITHM:**

- Step1: Start the program.
- Step2: Load the accumulator with the content of the memory location.
- Step3: Move the content of accumulator to register B.
- Step4: Load the accumulator with content of memory location.
- Step5: Add the content of A with register B.
- Step6: Store the content of accumulator in to memory location.
- Step7: Stop the program.

MNEMONICS:

```
MVI C,00H
LXI H,4200H
MOV A,M
INX H
MOV B,M
ADD B
JNC XX
INR C
XX STA 4202H
MOV A,C
STA 4203H
HLT
```

TABLE 1:

Memory	Label	Mnemonics		HEX	Description
		Instruction	Operand	CODE	
4100		MVI	C,00H	0E	Move the value 00 to reg C
4101				00	
4102		LXI	H,4200H	21	Load the value in HL pair.
4103				00	
4104				42	
4105		MOV	A,M	7E	Move the content of memory to reg A
4106		INX	H	23	Increment the memory location.
4107		MOV	B,M	46	Move the content of memory to reg B
4108		ADD	B	80	Add content of B reg to accumulator
4109		JNC	XX	D2	If there is no carry jump to XX
410A				0D	
410B				41	
410C		INR	C	0C	Increment value to reg c
410D	XX	STA	4202H	32	Store the content of accumulator in memory 4202
410E				02	
410F				42	
4110		MOV	A,C	79	Move the content of c reg to
4111		STA	4203H	32	Store the content of accumulator in memory 4203
4112				03	
4113				42	
4114		HLT		76	Halt the execution

OUTPUT (WITHOUT CARRY):**INPUT DATA:**

4200: 05

4201: 04

OUTPUT DATA:

4202: 09

4203: 00

OUTPUT (WITH CARRY):**INPUT DATA:**

4200: FF

4201: FF

OUTPUT DATA:

4202: FE

4203: 01

CONCLUSION:

Thus the addition of two 8-bit numbers using 8085 microprocessor was performed successfully

E & I DEPT.

Ex. No: 01b 8-BIT SUBTRACTION CONCEDERING WITH & WITHOUT CARRY.

AIM:

To perform subtraction of two 8-bit numbers using 8085 microprocessor.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- Power Supply

8-BIT SUBTRACTION:

ALGORITHM:

- Step1: Load the accumulator with content of memory location
 Step2: Move the content of accumulator to B reg
 Step3: Load the accumulator with the content of memory location
 Step4: Subtract the content of A with reg B
 Step5: Store the content of accumulator into memory location
 Step6: Stop the program

MNEMONICS:

```

MVI C,00
LXI H,4200H
MOV A,M
INX H
MOV B,M
SUB B
JNC XX
INR C
CMA
INR A
XX: STA 4202H
MOV A,C
STA 4203H
HLT
  
```

TABLE 2:

Memory	Label	Mnemonics		HEX CODE	Description
		Instruction	Operand		
4100		MVI	C,00	0E	Move the value 00 to reg c
4101				00	
4102		LXI	H,4200H	21	Load the value in HL reg pair
4103				00	
4104				42	
4105		MOV	A,M	7E	Move the content of memory to accumulator
4106		INX	H	23	Increment the HL register pair
4107		MOV	B,M	46	Move the content memory to reg B
4108		SUB	B	90	Sub reg B from accumulator
4109		JNC	XX	D2	Jump to label XX if no carry
410A				0F	
410B				41	
410C		INR	C	0C	Increment the C reg
410D		CMA		2F	Take 1's complement for accumulator
410E		INR	A	3C	Add 1's complement with 1
410F	XX	STA	4202H	32	Store the accumulator content in 4202H
4110				02	
4111				42	
4112		MOV	A,C	79	Move the content of C reg to A
4113		STA	4203H	32	Store the accumulator content in 4203H
				03	
				42	
4116		HLT		76	Halt the execution

OUTPUT (WITHOUT BORROW):**INPUT DATA:**

4200:05

4201:03

OUTPUT DATA:

4202:02

4203:00

OUTPUT (WITH BORROW):**INPUT DATA:**

4200:14

4201:89

OUTPUT DATA:

4202:75

4203:01

CONCLUSION:

Thus the subtraction of two 8-bit numbers using 8085 microprocessor was performed successfully

E & I DEPT.

Ex. No: 02a ADDITION OF TWO 16 BIT NUMBER WITH & WITHOUT CARRY.

AIM:

To write an assembly language program to add two 16-bit numbers using 8085 microprocessor kit.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- Power Supply

16-BIT ADDITION:

ALGORITHM:

- Step1: Start the program.
- Step2: Load 16 bit data in HL pair and move data 00H to 'C' register.
- Step3: Exchange data from HL pair to DE pair.
- Step4: Load another 16 bit data in HL pair.
- Step5: Add HL pair and DE pair contents and store the result in HL pair.
- Step6: If carry present Increment the content of CX register once else leave it as it is.
- Step7: Store data in HL pair to specified memory.
- Step8: Move data from 'C' register to accumulator.
- Step9: Store the accumulator data in specified memory.
- Step10: End

MNEMONICS:

```

MVI C, 00H
LHLD 5300
XCHG
LHLD 5302
DAD D
JNC Loop1
INR C
  
```

```

Loop1: SHLD 5500
      MOV A, C
      STA 5502
      HLT

```

TABLE:

Memory	Label	Mnemonics		HEX CODE	Description
		Instruction	Operand		
4100	Loop1:	MVI	C, 00H	0E	Move 00H to C register
4101				00	
4102		LHLD	5300	2A	Load 16 bit data to HL pair
4103				00	
4104				53	
4105		XCHG		EB	Exchange HL pair data with DE pair
4106		LHLD	5302	2A	Load another 16 bit data in HL pair
4107				02	
4108				53	
4109		DAD	D	19	Add HL pair and DE pair contents and store the result in HL pair
410A		JNC	Loop1	D2	If no carry move to specified address
410B				0E	
410C				41	
410D		INR	C	0C	Increment C register content once
410E		SHLD	5500	22	Store data in HL pair to specified
410F				00	memory
4110				55	
4111		MOV	A, C	79	Move 'C' register data to accumulator
4112		STA	5502	32	Store the accumulator data in
4113				02	specified memory
4114				55	
4115		HLT		76	Halt

OUTPUT:**INPUT DATA:**

5300: 77
5301: 88
5302: 99
5303: 11

OUTPUT DATA:

5500: 10
5501: 9A
5502: 00

CONCLUSION:

Thus an assembly language program to add two 16-bit numbers was written and executed using 8085 microprocessor kit.

E & I DEPT.

Ex. No: 02b SUBTRACTION OF TWO 16 BIT NUMBER WITH & WITHOUT CARRY.**AIM:**

To write an assembly language program to subtract two 16-bit numbers using 8085 microprocessor kit.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- Power Supply

ALGORITHM:

- Step1: Start the program.
- Step2: Move immediately the data 00H to C register and accumulator.
- Step3: Load 16 bit data in HL pair and exchange data from HL pair to DE pair.
- Step4: Load another 16 bit data in HL pair.
- Step5: Move data from 'L' register to accumulator.
- Step6: Subtract 'E' register content from accumulator.
- Step7: Move data from accumulator to 'L' register.
- Step8: Move data from 'H' register to accumulator.
- Step9: Subtract with borrow content of D register from accumulator.
- Step10: Jump to Step 11 when no carry increment the content of C register once.
- Step11: Move data from accumulator to 'H' register.
- Step12: Store data in HL pair to specified memory.
- Step13: Move data from 'C' register to accumulator.
- Step14: Store the accumulator data in specified memory.
- Step15: End.

MNEMONICS:

```
MVI C, 00H
MVI A, 00H
LHLD 5600
XCHG
LHLD 5602
```

MOV A, L
 SUB E
 MOV L, A
 MOV A, H
 SBB D
 JNC Loop1
 INR C
 MOV H, A
 Loop1: SHLD 5700
 MOV A, C
 STA 5702
 HLT

TABLE:

Memory	Label	Mnemonics		HEX CODE	Description
		Instruction	Operand		
41FE		MVI	C, 00H	0E	Move 00H to C register
41FF				00	
4200		MVI	A, 00H	3E	Move 00H to Accumulator
4201				00	
4202		LHLD	5600	2A	Load 16 bit data to HL pair
4203				00	
4204				56	
4205		XCHG		EB	Exchange HL pair data with DE pair
4206		LHLD	5602	2A	Load another 16 bit data in HL pair
4207				02	
4208				56	
4209		MOV	A, L	7D	Move 'L' register data to accumulator
420A		SUB	E	93	Subtract 'E' register content from accumulator
420B		MOV	L, A	6F	Move accumulator data to 'L' register
420C		MOV	A, H	7C	Move 'H' register data to Acc.
420D		SBB	D	9A	Subtract with borrow content of D register from accumulator
420E		JNC	Loop1	D2	If no carry move to specified address
420F				12	

4210				42	
4211		INR	C	0C	Increment C register content once
4212	Loop1:	MOV	H, A	67	Move Acc. data to 'H' register
4213		SHLD	5700	22	Store data in HL pair to specified
4214				00	memory
4215				57	
4216		MOV	A,C	79	Move 'C' register data to accumulator
4217		STA	5502	32	Store the accumulator data in
4218				02	specified memory
4219				57	
421A		HLT		76	Halt

OUTPUT:**INPUT DATA:**

5600: 11
 5601: 21
 5602: 77
 5603: 99

OUTPUT DATA:

5700: 66
 5701: 78

CONCLUSION:

Thus an assembly language program to subtract two 16-bit numbers was written and executed using 8085 microprocessor kit

E & I DEPT.

Ex. No: 03 MULTIPLICATION OF TWO 16 BIT NUMBERS.**AIM:**

To write an assembly language program to multiply two 16-bit data's using 8085 microprocessor kit.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- Power Supply

ALGORITHM:

Step1: Start the program.

Step2: Load 16 bit data in HL pair and move data from HL pair to Stack Pointer.

Step3: Load another 16 bit data in HL pair and move the data to DE pair.

Step4: Move data 0000H to BC and HL pair.

Step5: Add 16 bit data present in Stack Pointer with HL pair.

Step6: If carry present goto Step 8 else goto step 7.

Step7: Increment BC registers pair content once.

Step8: Decrement DE registers pair content once.

Step9: Move D registers content to accumulator and OR function it with E register content.

Step10: Check whether A is zero or not. If A=0 goto Step 6 else goto Step 5.

Step11: Store HL pair content in memory.

Step12: Move BC pair content to HL pair and then to memory.

Step13: End.

MNEMONICS:

LHLD 4200

SPHL

LHLD 4202

XCHG

LXI H, 0000H

LXI B, 0000H

Loop1: DAD SP

JNC Loop2

INX B

Loop2: DCX D

MOV A, D

ORA E

JNZ Loop1

SHLD 4500

MOV H, B

MOV L, C

SHLD 4502

HLT

TABLE:

Memory	Label	Mnemonics		HEX CODE	Description
		Instruction	Operand		
4100		LHLD	4200	2A	Load 16 bit data from memory to HL pair
4101				00	
4102				42	
4103		SPHL		F9	Move HL pair content to stack pointer
4104		LHLD	4202	2A	Load another 16 bit data from memory to accumulator
4105				02	
4106				42	

4107		XCHG		EB	Move HL pair content to DE pair
4108		LXI	H, 0000H	21	Move data 0000H to HL pair
4109				00	
410A				00	
410B		LXI	B, 0000H	01	Move data 0000H to BC pair
410C				00	
410D				00	
410E	Loop1:	DAD	SP	39	Add SP data with HL pair data
410F		JNC	Loop2	D2	If carry present jump to specified
4110				13	memory
4111				41	
4112		INX	B	03	Increment BC pair content once
4113	Loop2:	DCX	D	1B	Decrement DE pair content once
4114		MOV	A, D	7A	Move D register content to Acc.
4115		ORA	E	B3	OR function Accumulator content with E register content
4116		JNZ	Loop1	C2	Jump when no zero to specified
4117				0E	memory
4118				41	
4119		SHLD	4500	22	Store HL pair content in specified
411A				00	memory
411B				45	
411C		MOV	H, B	60	Move B register content to H register
411D		MOV	L, C	69	Move C register content to L register
411E		SHLD	4502	22	Store HL pair content in specified
411F				02	memory
4120				45	
4121		HLT		76	Halt

OUTPUT:**INPUT DATA:**

4200: 22

4201: 22

4202: 33

4203: 33

OUTPUT DATA:

4500: C6

4501: 92

4502: D3

4503: 06

CONCLUSION:

Thus an assembly language program to multiply two 16-bit data's and was written and executed using 8085 microprocessor kit.

Ex. No: 04 DIVISION OF TWO 8- BIT NUMBER**AIM:**

To write an assembly language program to divide two 8 bit data's using 8085 microprocessor kit.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- Power Supply

ALGORITHM:

Step1: Start the program.

Step2: Move immediately the data 00H to B register and load 8 bit data from memory to accumulator.

Step3: Move the accumulator content to C register.

Step4: Load another 8 bit data in HL pair.

Step5: Compare accumulator content with C register content. If equal Zero Flag gets affected.

Step6: If $A < C$ then carry gets affected.

Step7: Increment B register content once and subtract C register content from accumulator.

Step8: Goto Step 5.

Step9: Store the accumulator data in specified memory.

Step10: Move data from 'B' register to accumulator.

Step11: Store the accumulator data in specified memory.

Step12: End.

MNEMONICS:

MVI B, 00H

LDA 5100

MOV C, A

LDA 5100

Loop1: CMP C

```

JC Loop2
INR B
SUB C
JMP Loop1
Loop2: STA 5300
MOV A, B
STA 5301
HLT

```

TABLE:

Memory	Label	Mnemonics		HEX CODE	Description
		Instruction	Operand		
4200		MVI	B, 00H	06	Move immediately the data 00H to B register
4201				00	
4202		LDA	5100	3A	Load 8 bit data from memory to accumulator
4203				00	
4204				51	Move accumulator content to C register
4205		MOV	C,A	4F	
4206		LDA	5100	3A	Load another 8 bit data in HL pair from memory.
4207				01	
4208				51	Compare accumulator content with C register content
4209	Loop2:	CMP	C	B9	
420A		JC	Loop2	DA	When carry set jump to specified memory
420B				12	
420C				42	Increment B register content once
420D		INR	B	04	
420E		SUB	C	91	Subtract C register content from accumulator
420F		JMP	Loop1	C3	
4210				09	Jump to specified memory
4211				42	

4212	Loop1:	STA	5300	32	Store accumulator data in specified Memory
4213				00	
4214				53	
4215				78	
4215		MOV	A, B	78	Move data from 'B' register to accumulator
4216		STA	5301	32	Store accumulator data in specified memory
4217				01	
4218				53	
4219		HLT		76	Halt

OUTPUT:**INPUT DATA:**

5100: 20

5101: 60

OUTPUT DATA:

5300: 00

5301: 03

CONCLUSION:

Thus an assembly language program to divide two 8 bit data's was written and executed using 8085 microprocessor kit.

E & I DEPT.

Ex. No: 05a**SMALLEST AMONG 'N' NUMBERS****AIM:**

To find the smallest and largest among N numbers using 8085 microprocessor.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- Power Supply

SMALLEST AMONG N NUMBERS:**ALGORITHM:**

- Step1: Start the program
- Step2: Get the first number in the accumulator and move it to B
- Step3: Get the second number in the memory and move it to the accumulator
- Step4: Increment the address of memory and compare the data with accumulator
- Step5: If there is carry the above process is continued until carry is not present
- Step6: If carry is present then continue to compare with increment memory
- Step7: If carry is absent move that data to accumulator and decrement the B register until it become zero
- Step8: Store the smallest number in the accumulator
- Step9: End the program

MNEMONICS:

```

LDA 5000
MOV B,A
LXI H,5001
MOV A,M
XX:  INX H
      CMP M
      JC XX
      MOV A,M
YY:  DCR B

```

JNZ YY
STA 6000
HLT

TABLE:

Memory	Label	Mnemonics		HEX CODE	Description
		Instruction	Operand		
4500		LDA	5000	3A	Move the first data to accumulator
4501				00	
4502				50	
4503		MOV	B,A	47	Move the data from A to B
4504		LXI	H,5001	21	Move the second data to memory
4505				01	
4506				50	
4507		MOV	A,M	7E	Move data from M to A
4508	XX	INX	H	23	Increment the memory
4509		CMP	M	BE	Compare M with A
450A		JC	XX	DA	Jump if carry
450B				0E	
450C				45	
450D		MOV	A,M	7E	Move the data from M to A
450E	YY	DCR	B	05	Decrement B register
450F		JNZ	YY	C2	Jump if no zero
4510				08	
4511				45	
4512		STA	6000	32	Store the data in accumulator
4513				00	
4514				60	
4515		HLT		76	End of program

OUTPUT:**INPUT DATA:**

5000: 15

5001:03

5002:95

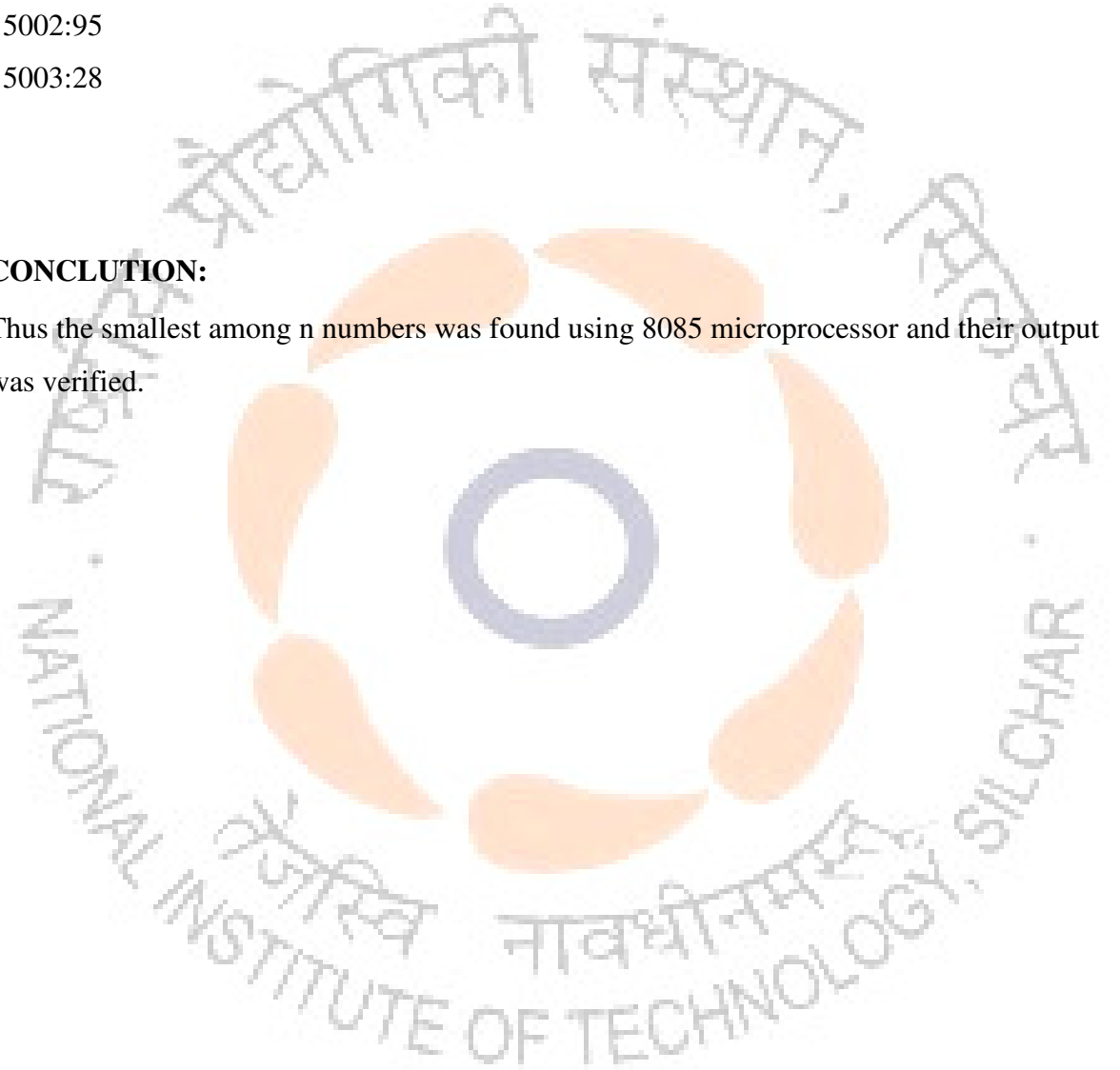
5003:28

OUTPUT DATA:

6000:03

CONCLUSION:

Thus the smallest among n numbers was found using 8085 microprocessor and their output was verified.



E & I DEPT.

Ex. No: 05b **FIND THE LARGEST NUMBER AMONG 'N' NUMBERS**

AIM:

To find the largest among N numbers using 8085 microprocessor.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- Power Supply

LARGEST AMONG N NUMBERS:

ALGORITHM:

Step1: Start the program

Step2: Get the first number in the accumulator and move it to B.

Step3: Get the second number in the memory H and move it to the accumulator.

Step4: Increment the address of memory and compare the data with accumulator.

Step5: If there is no carry the above process is continued until carry is present.

Step6: If carry is present move that data to accumulator and decrement the B register until it becomes zero.

Step7: Store the largest number in the accumulator.

Step8: End the program.

MNEMONICS:

LDA 5000

MOV B, A

LXI H,5001

MOV A, M

XX: INX H

CMP M

JNC XX

MOV A, M

YY: DCR B

JNZ YY
STA 6000
HLT

TABLE:

Memory	Label	Mnemonics		HEX CODE	Description
		Instruction	Operand		
4500		LDA	5000	3A	Move the first data to accumulator
4501				00	
4502				50	
4503		MOV	B,A	47	Move the data from A to B
4504		LXI	H,5001	21	Move the second data to memory
4505				01	
4506				50	
4507		MOV	A,M	7E	Move data from M to A
4508	XX	INX	H	23	Increment the memory
4509		CMP	M	BE	Compare M with A
450A		JNC	XX	DA	Jump no carry
450B				0E	
450C				45	
450D		MOV	A,M	7E	Move the data from M to A
450E	YY	DCR	B	05	Decrement B register
450F		JNZ	YY	C2	Jump if no zero
4510				08	
4511				45	
4512		STA	6000	32	Store the data in accumulator
4513				00	
4514				60	
4515		HLT		76	End of program

OUTPUT:**INPUT DATA:**

5000: 15
5001:03
5002:95
5003:28

OUTPUT DATA:

6000:95

CONCLUSION:

Thus the largest among n numbers was found using 8085 microprocessor and their output was verified.

E & I DEPT.

Ex. No: 06a**ASCENDING ORDER OF 'N' NUMBERS****AIM:**

To determine the ascending and descending order of the given number using 8085 microprocessor.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- Power Supply

ASCENDING ORDER:**ALGORITHM:**

Step1: Start the program

Step2: Get the first number and store it in B register and get the second number in memory and move it to accumulator.

Step3: Increment memory and compare it with accumulator if carry is present increment memory by decrementing B register if it is not zero.

Step4: If B register become zero decrement D register which contain number first, zero is not obtained then get in the memory.

Step5: If it is zero store the result in the accumulator.

Step6: If the compared output contains no carry, move the value in memory to C register and accumulator to memory and increment the value in memory.

Step7: stop the program.

MNEMONICS:

```
LDA 5000
MOV B, A
MOV D, A
MOV E, A
LOOP3 LXI H, 5001
      MOV A, M
```

```

MOV B, E
LOOP2 INX H
      CMP M
      JC  LOOP1
      MOV C, M
      MOV M, A
      DCX H
      MOV M, C
      INX H
LOOP1  MOV A, M
      DCR B;
      JNZ LOOP2
      DCR B
      JNZ LOOP3
      HLT

```

TABLE:

Memory	Label	Mnemonics		HEX CODE	Description
		Instruction	Operand		
4500		LDA	5000	3A	Get the first data to accumulator
4501				00	
4502				50	
4503		MOV	B,A	47	Move the data from A to B
4504		MOV	D,A	5F	Move the data from A to D
4505		MOV	E,A	57	Move the data from A to E
4506	LOOP 3	LXI	H,5001	21	Move second data to memory
4507				01	
4508				50	
4509		MOV	A,M	7E	Move M to Accumulator
450A		MOV	B,E	43	Move E to B register
450B	LOOP 2	INX	H	25	Increment H Register
450C		CMP	M	BE	Compare A and M
450D		JC	LOOP1	DA	Jump if carry to loop1
450E				15	

450F				45	
4510		MOV	C,M	4E	Move M to C register
4511		MOV	M,A	77	Move A to Memory
4512		DCX	H	2B	Decrement H Register
4513		MOV	M,C	71	Move the value from C to H
4514		INX	H	23	Increment H Register
4515	LOOP 1	MOV	A,M	7E	Move the value from M to A
4516		DCR	B	05	Decrement B Register
4517		JNZ	LOOP 2	C2	Jump is no zero to LOOP 2
4518				0B	
4519				45	
451A		DCR	D	15	Decrement D Register
451B		JNZ	LOOP 3	C2	Jump is no zero to LOOP 3
451C				06	
451D				45	
451E		HLT		76	End of Program

OUTPUT:**INPUT DATA:**

5000: 03

5001:05

5002:02

5003:06

OUTPUT DATA:

6000: 02

6001: 03

6002: 05

6003: 06

CONCLUSION:

Thus the Ascending order of given N- numbers was performed and their output was verified.

Ex. No: 06b**DECENDING ORDER OF 'N' NUMBERS****AIM:**

To determine the descending order of the given number using 8085 microprocessor.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- Power Supply

DESCENDING ORDER:**ALGORITHM:**

- Step1: Start the program
- Step2: Get the first number and store it in B register and get the second number in memory and move it to accumulator.
- Step3: Increment memory and compare it with accumulator if carry is present increment memory by decrementing B register if it is not zero.
- Step4: If B register become zero decrement D register which contain number first, zero is not obtained then get in the memory.
- Step5: If it is zero store the result in the accumulator.
- Step6: If the compared output contains no carry , move the value in memory to C register and accumulator to memory and increment the value in memory.
- Step7: Stop the program.

MNEMONICS:

```
LDA 5000
MOV B, A
MOV D, A
MOV E, A
LOOP3: LXI H, 5001
      MOV A, M
      MOV B, E
      LOOP2: INX H
```


CMP M
 JNC LOOP1
 MOV C, M
 MOV M, A
 DCX H
 MOV M, C
 INX H
 LOOP1: MOV A, M
 DCR B;
 JNZ LOOP2
 DCR B
 JNZ LOOP3
 HLT

TABLE:

Memory	Label	Mnemonics		HEX CODE	Description
		Instruction	Operand		
4500		LDA	5000	3A	Get the first data to accumulator
4501				00	
4502				50	
4503		MOV	B,A	47	Move the data from A to B
4504		MOV	D,A	5F	Move the data from A to D
4505		MOV	E,A	57	Move the data from A to E
4506	LOOP 3	LXI	H,5001	21	Move second data to memory
4507				01	
4508				50	
4509		MOV	A,M	7E	Move M to Accumulator
450A		MOV	B,E	43	Move E to B register
450B	LOOP 2	INX	H	25	Increment H Register
450C		CMP	M	BE	Compare A and M
450D		JNC	LOOP1	DA	Jump if carry to loop1
450E				15	
450F				45	

4510		MOV	C,M	4E	Move M to C register
4511		MOV	M,A	77	Move A to Memory
4512		DCX	H	2B	Decrement H Register
4513		MOV	M,C	71	Move the value from C to H
4514		INX	H	23	Increment H Register
4515	LOOP1	MOV	A,M	7E	Move the value from M to A
4516		DCR	B	05	Decrement B Register
4517		JNZ	LOOP 2	C2	Jump is no zero to LOOP 2
4518				0B	
4519				45	
451A		DCR	D	15	Decrement D Register
451B		JNZ	LOOP 3	C2	Jump is no zero to LOOP 3
451C				06	
451D				45	
451E		HLT		76	End of Program

OUTPUT:**INPUT DATA:**

5000: 03

5001:05

5002:02

5003:06

OUTPUT DATA:

6000: 06

6001: 05

6002: 03

6003: 02

CONCLUSION:

Thus the descending order of given N- numbers was performed and their output was verified.

Ex. No: 07a HEXADECIMAL TO DECIMAL CODE CONVERSIONS**AIM:**

To write an assembly language program to convert hexadecimal to decimal data's using 8085-microprocessor kit.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- Power Supply

HEXADECIMAL TO DECIMAL CONVERSION:**ALGORITHM:**

- Step1: Start the program.
- Step2: Load data from memory to accumulator and move the data 00 to D and E registers.
- Step3: Compare the accumulator data with the data 64.
- Step4: If carry=0 jump to Step 6 else jump to
- Step 5: Step5: Jump to Step 10.
- Step6: Subtract accumulator data by 64.
- Step7: Increment the content of D register once.
- Step8: If carry=0 jump to Step 6 else jump to Step 9.
- Step9: Decrement the content of D registers once and adds data 64 with accumulator.
- Step10: Subtract accumulator data by 0A and Increment E register content once.
- Step11: If carry=0 jump to Step 10 and Decrement E register content once.
- Step12: Add data 64 with accumulator and move it to C register.
- Step13: Move E registers content to accumulator.
- Step14: Rotate the accumulator content 4 times by left.
- Step15: Add C register content with accumulator content.
- Step16: Store data in accumulator pair to specified memory
- Step17: Move D registers content to accumulator

Step18: Store data in accumulator pair to specified memory.

Step19: End

MNEMONICS:

```

MVI  E, 00
MVI  D, 00
LDA  4200
CPI  64
JNC  Loop1
JMP  Loop2
Loop1: SUI  64
      INR  D
      JNC  Loop1
      DCR  D
      ADI  64
Loop2: SUI  0A
      INR  E
      JNC  Loop2
      DCR  E
      ADI  0A
      MOV  C, A
      MOV  A, E
      RLC
      RLC
      RLC
      RLC
      ADD  C
      STA  4500
      MOV  A, D
      STA  4501
      HLT
  
```

TABLE:

Memory	Label	Mnemonics		HEX CODE	Description
		Instruction	Operand		
4100		MVI	E, 00H	1E	Move data 00 to E register
4101				00	
4102		MVI	D, 00H	16	Move data 00 to D register
4103				00	
4104		LDA	4200	3A	Load data from memory to
4105				00	accumulator
4106				42	
4107		CPI	64	FE	Compare the accumulator data with
4108				64	the data 64
4109		JNC	410F	D2	If carry=0 jump to specified memory
410A				0F	
410B				41	
410C		JMP	4118	C3	Jump to specified memory
410D				18	
410E				41	
410F	Loop1	SUI	64	D6	Subtract accumulator data by 64
4110				64	
4111		INR	D	14	Increment D register content once
4112		JNC	410F	D2	If carry=0 jump to specified memory
4113				0F	
4114				41	
4115		DCR	D	15	Decrement D register content once
4116		ADI	64	C6	Add data 64 with accumulator
4117				64	
4118	Loop2	SUI	0A	D6	Subtract accumulator data by 0A
4119				0A	
411A		INR	E	1C	Increment E register content once
411B		JNC	4118	D2	If carry=0 jump to specified memory
411C				18	

411D				41	
411E		DCR	E	1D	Decrement E register content once
411F		ADI	0A	C6	Add data 64 with accumulator
4120				0A	
4121		MOV	C, A	4F	Move accumulator content to C register
4122		MOV	A, E	7B	Move E register content to accumulator
4123		RLC		07	
4124		RLC		07	Rotate the accumulator content 4 times
4125		RLC		07	by left
4126		RLC		07	
4127		ADD	C	81	Add C register content with accumulator content
4128		STA	4500	32	Store data in accumulator pair to
4129				00	specified memory
412A				45	
412B		MOV	A, D	7A	Move D register content to accumulator
412C		STA	4501	32	Store data in accumulator pair to
412D				01	specified memory
412E				45	
412F		HLT		76	Halt

OUTPUT:**INPUT DATA:**

4200: CE

OUTPUT DATA:

4500: 06

4501: 02

CONCLUSION:

Thus an assembly language program to convert hexadecimal to decimal data's was written and executed using 8085-microprocessor kit.

Ex. No: 07b HEXADECIMAL TO BINARY CODE CONVERSIONS

AIM:

To write an assembly language program to convert hexadecimal to binary data's using 8085-microprocessor kit.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- Power Chord

HEXADECIMAL TO BINARY CONVERSION:

ALGORITHM:

- Step1: Start the program.
- Step2: Load data from memory to accumulator
- Step3: Divide accumulator content by 2 and store the quotient in accumulator and remainder in next consecutive memory location.
- Step4: Repeat Step 3 until quotient becomes 1.
- Step5: If quotient becomes 1 store it in next memory location.
- Step6: End.

MNEMONICS:

```
LXI H, 4300
MOV A, M
MVI C, 02
```

Loop4: MVI D, 00

Loop1: SUB C

```
INR D
JC Loop2
JMP Loop1
```

Loop2: ADD C

```
INX H
MOV M, A
DCR D
MOV A, D
```

CPI 01
JZ Loop3
JMP Loop4

Loop3: INX H

MOV M, D

HLT

TABLE:

Memory	Label	Mnemonics		HEX CODE	Description
		Instruction	Operand		
4100		LXI	H,4300	21	Load memory to HL register pair
4101				00	
4102				43	
4103		MOV	A,M	72	Move data from memory to accumulator
4014		MVI	C,02	0E	Move data 02 to C register
4105				02	
4106	Loop4:	MVI	D, 00	16	Initialize D register
4107				00	
4108	Loop1:	SUB	C	91	Subtract C register content from A
4109		INR	D	14	Increment D register content once
410A		JC	Loop2	DA	Jump when carry=1 to specified
410B				10	Memory
410C				44	
410D		JMP	Loop1	C3	Jump to specified Memory
410E				08	
410F				41	
4110	Loop2:	ADD	C	81	Add C register content with A
4111		INX	H	23	Increment HL pair content once
4112		MOV	M, A	77	Move data from accumulator to memory
4113		DCR	D	15	Decrement D register content once
4114		MOV	A, D	7A	Move D register content to A

4115		CPI	01	FE	Compare D register content with 01
4116				01	
4117		JZ	Loop3	C4	Jump when ZF=1 to specified
4118				1C	Memory
4119				41	
411A		JMP	Loop4	C3	Jump to specified Memory
411B				06	
411C				44	
411D	Loop3:	INX	H	23	Increment HL pair memory once
411E		MOV	M, D	72	Move D register data to Memory
411F		HLT		76	Halt

OUTPUT:**INPUT DATA:**

4300: DE

OUTPUT DATA:

4301: 00

4305: 01

4302: 01

4306: 00

4303: 01

4307: 01

4304: 01

4308: 01

CONCLUSION:

Thus an assembly language program to convert hexadecimal to binary data's was written and executed using 8085-microprocessor kit.

E & I DEPT.

Ex. No: 08**FIBONACCI SERIES****AIM:**

To write an assembly language program to generate Fibonacci series of 'N' number of data's using 8085 microprocessor kit.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- Power Supply

ALGORITHM:

- Step1: Start the program.
- Step2: Move data 0A to 'B' register.
- Step3: Clear the accumulator content and load data of 4200 to HL pair.
- Step4: Move the data from accumulator to specified memory.
- Step5: Decrement 'B' content once and Increment accumulator content once.
- Step6: Increment 'HL' pair content once.
- Step7: Move the data from accumulator to specified memory and decrement 'HL' pair content once.
- Step8: Move data in memory to 'C' register and add 'C' register content with Acc.
- Step9: Increment 'HL' pair content once and Decrement 'B' content once.
- Step10: If no zero goto Step 11 else goto Step 6.
- Step11: End.

MNEMONICS:

MVI B, 0A

XRA A

LXI H, 4200

MOV M, A

DCR B

INR A

Loop1: INX H

MOV M, A

DCX H
 ADD M
 INX H
 DCR B
 JNZ Loop1
 HLT

TABLE:

Memory	Label	Mnemonics		HEX CODE	Description
		Instruction	Operand		
4100		MVI	B, 0A	06	Move data 0A to 'B' register content
4101				0A	
4102		XRA	A	AF	Clear the accumulator content
4103		LXI	H, 4200	21	Load data of 4200 to HL pair
4014				00	
4105				42	
4106		MOV	M, A	77	Move data from accumulator to M
4107		DCR	B	05	Decrement 'B' content once
4108		INR	A		Increment accumulator content once
4109	Loop1	INX	H	23	Increment H register content once
410A		MOV	M, A	77	Move data from accumulator to M
410B		DCX	H	2B	Decrement 'HL' pair content once
410C		ADD	M	86	Add data from M with accumulator
410D		INX	H	23	Increment 'HL' pair content once
410E		DCR	B	05	Decrement 'B' content once
410F		JNZ	Loop1	CZ	Jump to specified memory if no zero
4110				09	
4111				41	
4112		HLT		76	Halt

OUTPUT:**INPUT DATA:**

4101: 0A

OUTPUT DATA:

4200: 00

4205: 05

4201: 01

4206: 08

4202: 01

4207: 13

4203: 02

4208: 1B

4204: 03

4209: 2E

CONCLUSION:

Thus an assembly language program to generate Fibonacci series of 'N' number of data's was written and executed using 8085 microprocessor kit.

E & I DEPT.

Ex. No: 09**FACTORIAL OF 'N' DATA'S****AIM:**

To write an assembly language program to calculate factorial of N number of data's using 8085 microprocessor kit.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- Power Chord

ALGORITHM:

- Step1: Start the program.
- Step2: Load 16 bit data in HL pair and move data from HL pair to DE pair.
- Step3: Move E register content to B register.
- Step4: Decrement B register content once and move B register content to C register.
- Step5: Decrement B register pair content once and load 0000H HL pair.
- Step6: Add DE pair content with HL pair content.
- Step7: Decrement B register content once.
- Step8: If there is no zero flag jump to Step 6.
- Step9: Move HL pair content to DE pair and load 0000H HL pair.
- Step10: Move C register content to B register.
- Step11: Decrement C register content once.
- Step12: If zero flag is set jump to Step 18.
- Step13: Jump to Step 9.
- Step14: Move HL pair content to DE pair.
- Step15: Store HL pair content in specified memory.
- Step16: End

MNEMONICS:

LHLD 4200

XCHG

MOV B, E

```

DCR B
MOV C, B
DCR C
LXI H, 0000H
Loop1: DAD D
      DCR B
      JNZ Loop1
      XCHG
      LXI H, 0000H
      MOV B, C
      DCR C
      JZ   Loop2
      JMP Loop1
Loop2: XCHG
      SHLD 4300
      HLT

```

TABLE:

Memory	Label	Mnemonics		HEX CODE	Description
		Instruction	Operand		
4100		LHLD	4200	2A	Load 16 bit data to HL pair
4101				00	
4102				42	
4103		XCHG		EB	Exchange HL pair data with DE pair
4014		MOV	B, E	42	Move 'E' register data to 'B' register
4105		DCR	B	05	Decrement 'B' content once
4106		MOV	C, B	48	Move 'B' register data to 'C' register
4107		DCR	C	0D	Decrement 'C' content once
4108		LXI	H, 0000H	21	Load data of 0000 to HL pair
4109				00	
410A				00	
410B	Loop1	DAD	D	19	Add HL pair and DE pair contents and store the result in HL pair

410C		DCR	B	05	Decrement 'B' content once
410D		JNZ	410B	C2	Jump when no zero to specified
410E				0B	address
410F				41	
4110		XCHG		4B	Exchange HL pair data with DE pair
4111		LXI	H, 0000H	21	Load data of 0000 to HL pair
4112				00	
4113				00	
4114		MOV	B, C	41	Move 'C' register data to 'B' register
4115		DCR	C	0D	Decrement 'C' content once
4116		JZ	411C	CA	If zero flag set jump to specified
4117				1C	address
4118				41	
		JMP	410B	C3	Jump to specified address
4119					
411A				0B	
411B				41	
411C	Loop2	XCHG		EB	Exchange HL pair data with DE pair
411D		SHLD	4300	22	Store data in HL pair to specified
411E				00	memory
411F				43	
4120		HLT		76	Halt

OUTPUT

INPUT DATA:

4200: 05

OUTPUT DATA:

4300: 78

CONCLUSION: Thus an assembly language program to calculate factorial of N number of data's was written and executed using 8085 microprocessor kit.

Ex. No: 10**SUM OF SERIES OF 'N' NUMBER****AIM:**

To write an assembly language program to calculate sum of series of 'N' number of data's with carry using 8085 microprocessor kit.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- Power Chord

ALGORITHM:

- Step1: Start the program.
- Step2: Move the memory address to 'H' register.
- Step3: Move the data present in memory to 'E' register.
- Step4: Initialize the 'C' register as 00H.
- Step5: Clear the accumulator content and increment 'H' register pair.
- Step6: Move the data from memory to accumulator
- Step7: Increment 'H' content once and add it with accumulator.
- Step8: Decrement 'E' content once. Check whether carry is present or not.
- Step9: If no carry, increment 'H' content once and add it with accumulator.
- Step10: If carry present Increment the content of CX register once and repeat Step 8.
- Step11: Repeat the above steps for 'N' number of data's.
- Step12: Store the result in specified memory.
- Step13: End.

MNEMONICS:

LXI H, 5300

MOV E, M

MVI C, 00H

XRA A

Loop1: INX H

ADD M

JNC Loop 2

INR C

Loop2: DCR E

JNZ Loop1

STA 4600

MOV A, C

STA 4601

HLT

TABLE:

Memory	Label	Mnemonics		HEX CODE	Description
		Instruction	Operand		
4100	Loop1	LXI	H, 5300	21	Load 16 bit address in HL register pair
4101				00	
4102				53	
4103		MOV	E, M	5E	Move data from memory to E register
4014		MVI	C, 00H	0E	Move data 00H to register 'C'
4105				00	
4106		XRA	A	AF	OR function the accumulator content
4107		INX	H	23	Increment H register content once
4108		ADD	M	86	Add data from M with accumulator
4109		JNC	Loop 2	D2	Jump on no carry
410A	Loop2:			0D	
410B				41	
410C		INR	C	0C	Increment C register content once
410D		DCR	E	1D	Decrement 'E' content once
410E		JNZ	Loop1	C2	Jump on no zero
410F				07	
4110				41	
4111		STA	4600	32	Store data from accumulator to the
4112				00	specified memory
4113				46	
4114		MOV	A, C	79	Move data form C register to Acc.
4115		STA	4601	32	Store data from accumulator to the
4116				01	specified memory
4117				46	
4118		HLT		76	Halt

OUTPUT:**INPUT DATA:**

5300: 0A 5306: 06
5301: 11 5307: 17
5302: 12 5308: 09
5303: 13 5309: 0A
5304: 04 530A: 01
5305: 05

OUTPUT DATA:

4600: 70
4601: 00

CONCLUSION:

Thus an assembly language program to calculate sum of series of 'N' number of data's with carry was written and executed using 8085 microprocessor kit. Manual

E & I DEPT.



8086 MICROPROCESSOR PROGRAMMING

E & I DEPT.

Ex. No: 11A ADDITION OF TWO 32 BIT NUMBER**AIM:**

To write an assembly language program to add two 32-bit numbers using 8086 microprocessor kit.

APPARATUS REQUIRED:

- 8086 Microprocessor Kit
- Power Supply

32 - BIT ADDITION:**ALGORITHM:**

Step1: Start the program.

Step2: Move immediately the number 0000H to CX register.

Step3: Copy the contents of the memory 3000 to AX register.

Step4: Add the content of the memory 3004 with the content of AX register.

Step5: Copy the content to AX register to two memories from 2000.

Step6: Copy the contents of the memory 3002 to AX register.

Step7: Add the content of the memory 3006 with the content of AX register.

Step8: Jump to specified memory location if there is no carry i.e. CF=0.

Step9: Increment the content of CX register once.

Step10: Copy the content to AX register to two memories from 2002.

Step11: Copy the content to CX register to two memories from 2004.

Step12: End

MNEMONICS:

MOV CX, 0000

MOV AX, [3000]

ADD AX, [3004]

MOV [2000], AX

MOV AX, [3002]

ADC AX, [3006]

JNC loop1

INC CX

Loop1 MOV [2002], AX

MOV [2004], CX

HLT

TABLE:

Memory	Label	Mnemonics		Description
		Instruction	Operand	
1000		MOV	CX,0000	Move immediately 0000H to CX register
1004		MOV	AX, [3000]	Copy contents of 3000 to AX register
1008		ADD	AX, [3004]	Add content of memory 3004 with content of AX register
100C		MOV	[2000], AX	Copy content to AX register to two memories from 2000
1010		MOV	AX, [3002]	Copy contents of memory 3002 to AX register
1014		ADC	AX, [3006]	Add content of memory 3006 with content of AX register
1018		JNC	loop1	Jump to specified memory CF=0
101A		INC	CX	Increment content of CX register once
101B	Loop1	MOV	[2002], AX	Copy content to AX register to two memories from 2002
101F		MOV	[2004], CX	Copy content to CX register to two memories from 2004
1023		HLT		Halt

OUTPUT:**INPUT DATA:**

3000: 9999
3002: 9999
3004: 9999
3006: 9999

OUTPUT DATA:

2000: 3332
2002: 3333
2004: 1

CONCLUSION:

Thus an assembly language program to add two 32-bit numbers was written and executed using 8086 microprocessor kit.

E & I DEPT.

Ex. No: 11B SUBTRACTON OF TWO 32 BIT NUMBER**AIM:**

To write an assembly language program to subtract two 32-bit numbers using 8086 microprocessor kit.

APPARATUS REQUIRED:

- 8086 Microprocessor Kit
- Power Supply

32 - BIT SUBTRACTION:**ALGORITHM:**

- Step1: Start the program.
- Step2: Move immediately the number 0000H to CX register.
- Step3: Copy the contents of the memory 3000 to AX register.
- Step4: Add the content of the memory 3004 with the content of AX register.
- Step5: Copy the content to AX register to two memories from 2000.
- Step6: Copy the contents of the memory 3002 to AX register.
- Step7: Subtract the content of the memory 3006 from AX register.
- Step8: Jump to specified memory location if there is no carry i.e. CF=0.
- Step9: Increment the content of CX register once.
- Step10: Copy the content to AX register to two memories from 2002.
- Step11: Copy the content to CX register to two memories from 2004.
- Step12: End.

MNEMONICS:

```
MOV CX, 0000
MOV AX, [3000]
ADD AX, [3004]
MOV [2000], AX
MOV AX, [3002]
```

SBB AX, [3006]
 JNC loop1
 INC CX
 Loop1 MOV [2002], AX
 MOV [2004], CX
 HLT

TABLE:

Memory	Label	Mnemonics		Description
		Instruction	Operand	
1000		MOV	CX,0000	Move immediately 0000H to CX register
1004		MOV	AX, [3000]	Copy contents of 3000 to AX register
1008		ADD	AX, [3004]	Add content of memory 3004 with content of AX register
100C		MOV	[2000], AX	Copy content to AX register to two memories from 2000
1010		MOV	AX, [3002]	Copy contents of memory 3002 to AX register
1014		SBB	AX, [3006]	Subtract content of memory 3006 from content of AX register
1018		JNC	loop1	Jump to specified memory CF=0
101A		INC	CX	Increment content of CX register once
101B	Loop1	MOV	[2002], AX	Copy content to AX register to two memories from 2002
101F		MOV	[2004], CX	Copy content to CX register to two memories from 2004
1023		HLT		Halt

OUTPUT:**INPUT DATA:**

3000: 9999
3002: 9799
3004: 9999
3006: 9999

OUTPUT DATA:

2000: 0000
2002: FE00

CONCLUSION:

Thus an assembly language program to subtract two 32-bit numbers was written and executed using 8086 microprocessor kit.

E & I DEPT.

Ex. No: 12A 16 BIT MULTIPLICATION**AIM:**

To write an assembly language program to multiply two unsigned 16-bit numbers using 8086 microprocessor kit.

APPARATUS REQUIRED:

- 8086 Microprocessor Kit
- Power Supply

MULTIPLICATION:**ALGORITHM:**

Step 1: Start the program.

Step2: Copy the contents of the memory 3000 to AX register.

Step3: Copy the contents of the memory 3002 to CX register.

Step4: Multiply the content of the CX register with the content of accumulator.

Step5: Copy the content to AX register to the memory 2000.

Step6: Copy the contents of DX register to the memory 2002.

Step7: End.

MNEMONICS:

MOV AX, [3000]

MOV CX, [3002]

MUL CX

MOV [2000], AX

MOV [2002], DX

HLT

TABLE:

Memory	Label	Mnemonics		Description
		Instruction	Operand	
1000		MOV	AX, [3000]	Copy contents of 3000 to AX register
1004		MOV	CX, [3002]	Copy contents of 3002 to CX register
1008		MUL	CX	Multiply the content of the CX register with the content of accumulator
100A		MOV	[2000], AX	Copy content to AX register to the memory 2000
100E		MOV	[2004], DX	Copy content to DX register to the memory 2002
1012		HLT		Halt

OUTPUT:**INPUT DATA:**

3000: 1234

3002: 5678

OUTPUT DATA:

2000: 0060

2002: 0626

CONCLUSION:

Thus an assembly language program to multiply two unsigned 16-bit numbers was written and executed using 8086 microprocessor kit.

E & I DEPT.

Ex. No: 12B 16 BIT DIVISION**AIM:**

To write an assembly language program to divide two unsigned 16-bit numbers using 8086 microprocessor kit.

APPARATUS REQUIRED:

- 8086 Microprocessor Kit
- Power Supply

DIVISION:**ALGORITHM:**

Step 1: Start the program.

Step2: Copy the contents of the memory 3000 to AX register.

Step3: Copy the contents of the memory 3002 to CX register.

Step4: Divide the content of the CX register from the content of accumulator.

Step5: Copy the content to AX register to the memory 2000.

Step6: Copy the contents of DX register to the memory 2002.

Step7: End.

MNEMONICS:

MOV AX, [3000]

MOV CX, [3002]

DIV CX

MOV [2000], AX

MOV [2002], DX

HLT

TABLE:

Memory	Label	Mnemonics		Description
		Instruction	Operand	
1000		MOV	AX, [3000]	Copy contents of 3000 to AX register
1004		MOV	CX, [3002]	Copy contents of 3002 to CX register
1008		DIV	CX	Divide the content of the CX register with the content of accumulator
100A		MOV	[2000], AX	Copy content to AX register to the memory 2000
100E		MOV	[2004], DX	Copy content to DX register to the memory 2002
1012		HLT		Halt

OUTPUT:**INPUT DATA:**

3000: 1234

3002: 5678

OUTPUT DATA:

2000: 0000

2002: 4444

CONCLUSION:

Thus an assembly language program to divide two unsigned 16-bit numbers was written and executed using 8086 microprocessor kit.

Ex. No: 13 FACTORIAL OF 'N' NUMBER**AIM:**

To write an assembly language program to calculate factorial of n-numbers using 8086 microprocessor kit.

APPARATUS REQUIRED:

- 8086 Microprocessor Kit
- Power Supply

ALGORITHM:

Step 1: Start the program.

Step2: Move immediately the number 0000H to AX register.

Step3: Copy the contents of the memory 3000 to CX register.

Step4: Move immediately the number 0001H to AX register.

Step5: Multiply the content of the CX register with the content of accumulator.

Step6: Decrement the content of CX register once.

Step7: Jump to specified memory location if there is no zero in CX register.

Step8: Copy the content to AX register to two memories from 2000.

Step9: End

MNEMONICS:

MOV AX, 0001

MOV CX, [3000]

MOV AX, 0001

Loop1 MUL CX

DEC CX

JNZ loop1

MOV [2000], AX

HLT

TABLE:

Memory	Label	Mnemonics		Description
		Instruction	Operand	
1000		MOV	AX, 0001	Move immediately the number 0001H to AX register
1004		MOV	CX, [3000]	Copy the contents of memory 3000 to CX register
1006		MOV	AX, 0001	Move immediately the number 0000H to AX register
100A	loop1	MUL	CX	Multiply content of CX register with content of accumulator
100B		DEC	CX	Decrement content of CX register once
100C		JNZ	loop1	Jump to specified memory location if there is no zero in CX register
100E		MOV	[2000], AX	Copy content to AX register to memory 2000
1012		HLT		Halt

OUTPUT:**INPUT DATA:**

3000: 0008

OUTPUT DATA:

2000: 9d80

CONCLUSION:

Thus an assembly language program to calculate factorial of n-numbers was written and executed using 8086 microprocessor kit.

Ex. No: 14**SORTING IN ASCENDING ORDER****AIM:**

To write an assembly language program to sort n-numbers in ascending order using 8086 microprocessor kit.

APPARATUS REQUIRED:

- 8086 Microprocessor Kit
- Power Supply

ALGORITHM:

- Step 1: Start the program.
- Step2: Load data's into the memory.
- Step3: Set the conditions to sort n-numbers in ascending order.
- Step4: Sort the 'n' given numbers in ascending order.
- Step5: Store the result in the memory.
- Step6: Display the sorted result from memory.
- Step7: End.

MNEMONICS:

```
MOV BX, 2000
MOV CX, [BX]
MOV CH, CL
```

```
Loop2 INC BX
      INC BX
      MOV AX, [BX]
      INC BX
      INC BX
      CMP AX, [BX]
      JC  loop1
      MOV DX, [BX]
      MOV [BX], AX
      DEC BX
```



```

DEC BX
MOV [BX], DX
INC BX
INC BX
Loop1 DEC BX
      DEC BX
      DEC CL
      JNZ loop2
      MOV BX, 2000
      MOV CH, CL
      DEC CH
      JNZ loop2
      HLT

```

TABLE:

Memory	Label	Mnemonics		Description
		Instruction	Operand	
1000		MOV	BX, 2000	Move 2000 to BX register
1004		MOV	CX, [BX]	Move BX memory data to CX register
1006		MOV	CH, CL	Move data from CL to CH
1008	Loop2	INC	BX	Increment BX register content once
1009		INC	BX	Increment BX register content once
100A		MOV	AX, [BX]	Move BX memory data to AX register
100C		INC	BX	Increment BX register content once
100D		INC	BX	Increment BX register content once
100E		CMP	AX, [BX]	Compare AX register content and BX memory
1011		JC	loop1	Jump to specified memory location if carry is 1
1013		MOV	DX, [BX]	Move BX memory data to DX register
1015		MOV	[BX], AX	Move data from AX register to BX memory data

1017		DEC	BX	Decrement BX register content once
1018		DEC	BX	Decrement BX register content once
1019		MOV	[BX], DX	Move data from DX register to BX memory data
101B		INC	BX	Increment BX register content once
101C		INC	BX	Increment BX register content once
101D	Loop1	DEC	BX	Decrement BX register content once
101E		DEC	BX	Decrement BX register content once
101F		DEC	CL	Decrement CL register content once
1020		JNZ	loop2	Jump to specified memory location if there is no zero in CX register
1022		MOV	BX, 2000	Move 2000 to BX register
1026		MOV	CH, CL	Copy CL register data to CH register
1028		DEC	CH	Decrement CH register content once
1029		JNZ	loop2	Jump to specified memory location if there is no zero in CX register
102B		HLT		Halt

OUTPUT:**INPUT****OUTPUT**

2000: 05

2000: 01

2002: 03

2002: 02

2004: 01

2004: 03

2006: 04

2006: 04

2008: 02

2008: 05

CONCLUSION:

Thus an assembly language program to sort n-numbers in ascending order was written and executed using 8086 microprocessor kit.

Ex No: 15 SUM OF N NUMBERS IN AN ARRAY**AIM:**

To write a program to find sum of 'n' numbers in an array.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- Power Chord

ALGORITHM:

- Step1: Start the program.
Step2: Initialize the counter.
Step3: Get the first number.
Step4: Decrement the counter.
Step5: Load the base address of an array in to BX
Step6: By using the loop get the next number in to DX and add it with AX.
Step7: Increment the pointer and decrement the counter.
Step8: If the counter value is not equal to zero then go to step6
Step9: Else store the result.
Step10: Stop the program.

MNEMONICS:

```
MOV CL, [2000]
MOV AX, [2002]
DEC CL
XOR D1, D1
LEA BX, [2004]
LOOP1 MOV DX, [BX+D1]
      ADD AX, BX
      INC D1
      INC D1
      DEC CL
```

```

JNZ LOOP1
MOV [3000], AX
HLT

```

TABLE:

LABEL	OPCODE	OPERAND	DESCRIPTION
	MOV	CL,[2000]	Move the memory content to CL.
	MOV	AX,[2002]	Move the memory content to AX
	DEC	CL	Decrement the CL register.
	XOR	D1,D1	XOR,D1 registers
	LEA	BX,[2004]	Move the content of 2004 to BX
	MOV	DX,[BX+DI]	Move the content of BX+D1 to DX
	ADD	AX,BX	Add AX with DX content.
	INC	DI	Increment D1
	INC	DI	Increment D1
	DEC	CL	Decrement CL
LOOP 1	JNZ	LOOP 1	If zero flag is re-set go to loop1
	MOV	[3000],AX	Move the content to memory location
	HLT		Halt

OUTPUT:**INPUT DATA:**

2000:0003

2002:0002

2004:0003

2006:0001

OUTPUT DATA:

3000:0006

CONCLUSION:

Thus the sum of n numbers in an array has been done using 8086 microprocessor and the output is verified.



8051 MICROCONTROLLER PROGRAMMING

E & I DEPT.

Ex. No: 16**16 BIT ADDITION****AIM:**

To write an assembly language program to add the two 16 bit data's using 8051 Micro controller.

APPARATUS REQUIRED:

- 8051 Microcontroller kit.
- Power Supply.

ALGORITHM:

- Step1: Start the program.
- Step2: Load the lower byte of the two data's into accumulator and R0 register.
- Step3: Add the two data's.
- Step4: Move the added data into R6 register and initialize the R2 register.
- Step5: Load the higher byte of the two data's into accumulator and R1 register.
- Step6: Add the two data's with carry.
- Step7: If carry comes, increment R2 registers content once.
- Step8: Store the accumulator data and R6 and R2 register data's into the memory.
- Step9: Stop the program

MNEMONICS:

```

MOV    DPTR,#4400
MOVX   A,@DPTR
MOV    R0,A
MOV    R2,#00
INC    DPTR
MOVX   A,@DPTR
MOV    R1,A
INC    DPTR
MOVX   A,@DPTR
ADD    A,R0

```

```

MOV      R6,A
INC      DPTR
MOVX     A,@DPTR
ADDC     A,R1
JNC      LOOP1
INC      R2
LOOP1:   INC      DPTR
MOVX     @DPTR,A
INC      DPTR
MOV      A,R6
MOVX     @DPTR,A
INC      DPTR
MOV      A,R2
MOVX     @DPTR,A
LOOP2:   SJMP     LOOP2

```

TABLE:

Memory	Label	MNEMONICS	Hex code	Description
4100		MOV DPTR,#4400	90	Move data 4400 to DPTR
4101			44	
4102			00	
4103		MOVX A,@DPTR	E0	Move data from DPTR to Accumulator.
4104		MOV R0,A	F8	Move data from Accumulator to R0 register.
4105		MOV R2,#00	7A	Clear the R2 register.
4106			00	
4107		INC DPTR	A3	Increment DPTR content once.
4108		MOVX A,@DPTR	E0	Load the data from DPTR to Accumulator.
4109		MOV R1,A	F9	Move the data to R1 register from Accumulator.

410A		INC DPTR	A3	Increment DPTR content once.
410B		MOVX A,@DPTR	E0	Load the data from DPTR to Accumulator.
410C		ADD A,R0	28	Add Accumulator data and R0 register data.
410D		MOV R6,A	FE	Move data from Accumulator to R6 register.
410E		INC DPTR	A3	Increment DPTR content once.
410F		MOVX A,@DPTR	E0	Load the data from DPTR to Accumulator.
4110		ADDC A,R1	39	Add Accumulator data and R0 register data with carry.
4111			50	
4112		JNC LOOP1	01	Jump when carry=0 to loop1.
4113		INC R2	0A	Increment the content of R2 register once.
4114	Loop1	INC DPTR	A3	Increment DPTR content once.
4115		MOVX @DPTR,A	F0	Store the Accumulator data to DPTR.
4116		INC DPTR	A3	Increment DPTR content once.
4117		MOV A,R6	EE	Move the data from R6 register to Accumulator.
4118		MOVX @DPTR,A	F0	Store the Accumulator data to DPTR.
4119		INC DPTR	A3	Increment DPTR content once.
411A		MOV A,R2	EA	Move the data from R2 register to Accumulator.
411B		MOVX @DPTR,A	F0	Store the Accumulator data to DPTR.
411C	Loop2	SJMP LOOP2	80	Jump to loop2.
411D			41	
411E			1C	

OUTPUT:**INPUT DATA:**

4400: 23

4401: 32

4402: 47

4403: 74

OUTPUT DATA:

4404: A6

4405: 6A

4406: 00

CONCLUSION:

Thus an assembly language program to add two 16-bit data's was written and executed using 8051 micro controller kit.

Ex. No: 17 16 BIT SUBTRACTION

AIM:

To write an assembly language program to subtract the two 16 bit data's using 8051 Micro controller.

APPARATUS REQUIRED:

- 8051 Microcontroller kit.
- Power Supply.

ALGORITHM:

- Step1: Start the program.
- Step2: Load the lower byte of the two data's into accumulator and R0 register.
- Step3: Subtract the two data's.
- Step4: Move the subtracted data into R6 register and initialize the R2 register.
- Step5: Load the higher byte of the two data's into accumulator and R1 register.
- Step 6: Subtract the two data's with borrow.
- Step7: If carry comes, increments R2 register content once.
- Step8: Store the accumulator data and R6 and R2 register data's into the memory.
- Step9: Stop the program.

MNEMONICS:

```

MOV      DPTR,#4400
MOVX     A,@DPTR
MOV      R0,A
MOV      R2,#00
INC      DPTR
MOVX     A,@DPTR
MOV      R1,A
INC      DPTR
MOVX     A,@DPTR

```

```

        SUBB    A,R0
        MOV     R6,A
        INC     DPTR
        MOVX    A,@DPTR
        SUBB    A,R1
        JNC     LOOP1
        INC     R2
LOOP1:  INC     DPTR
        MOVX    @DPTR,A
        INC     DPTR
        MOV     A,R6
        MOVX    @DPTR,A
        INC     DPTR
        MOV     A,R2
        MOVX    @DPTR,A
LOOP2:  SJMP    LOOP2

```

TABLE:

Memory	Label	Mnemonics	Hex code	Description
4100		MOV DPTR,#4400	90	Move data 4400 to DPTR
4101			44	
4102			00	
4103		MOVX A,@DPTR	E0	Move data from DPTR to Accumulator.
4104		MOV R0,A	F8	Move data from Accumulator to R0 register.
4105		MOV R2,#00	7A	Clear the R2 register.
4106			00	
4107		INC DPTR	A3	Increment DPTR content once.
4108		MOVX A,@DPTR	E0	Load the data from DPTR to Accumulator.

4109		MOV R1,A	F9	Move the data to R1 register from Accumulator.
410A		INC DPTR	A3	Increment DPTR content once.
410B		MOVX A,@DPTR	E0	Load the data from DPTR to Accumulator.
410C		SUBB A,R0	98	Subtract Accumulator data and R0 register data.
410D		MOV R6,A	FE	Move data from Accumulator to R6 register.
410E		INC DPTR	A3	Increment DPTR content once.
410F		MOVX A,@DPTR	E0	Load the data from DPTR to Accumulator.
4110		SUBB A,R1	99	Subtract Accumulator data and R0 register data with carry.
4111			50	
4112		JNC LOOP1	01	Jump when carry=0 to loop1.
4113		INC R2	0A	Increment the content of R2 register once.
4114	Loop1	INC DPTR	A3	Increment DPTR content once.
4115		MOVX @DPTR,A	F0	Store the Accumulator data to DPTR.
4116		INC DPTR	A3	Increment DPTR content once.
4117		MOV A,R6	EE	Move the data from R6 register to Accumulator.
4118		MOVX @DPTR,A	F0	Store the Accumulator data to DPTR.
4119		INC DPTR	A3	Increment DPTR content once.
411A		MOV A,R2	EA	Move the data from R2 register to Accumulator.
411B		MOVX @DPTR,A	F0	Store the Accumulator data to DPTR.
411C	Loop2	SJMP LOOP2	80	Jump to loop2.
411D			41	
411E			1C	

OUTPUT:**INPUT DATA:**

4500: BC
4501: 19
4502: 88
4503: 99

OUTPUT DATA:

4504: 80
4505: 34
4506: 01

CONCLUSION:

Thus an assembly language program to subtract two 16-bit data's was written and executed using 8051 micro controller kit.

E & I DEPT.

Ex. No: 18**16 BIT MULTIPLICATION****AIM:**

To write an assembly language program to multiply two 16 bit data's using 8051 Micro controller.

APPARATUS REQUIRED:

- 8051 Microcontroller kit.
- Power Supply.

ALGORITHM:

Step1: Start the program.

Step2: Load the two data's into Accumulator and B register.

Step3: Multiply the two data's.

Step4: Store the result into the memory.

Step5: Stop the program.

MNEMONICS:

```

MOV      DPTR,#4400
MOVX     A,@DPTR
MOV      0F0,A
INC      DPTR
MOVX     A,@DPTR
MUL      AB
INC      DPTR
MOVX     @DPTR,A
INC      DPTR
MOV      A,0F0
MOVX     @DPTR,A
LOOP1:   SJMP    LOOP1
  
```

TABLE:

Memory	Label	Mnemonics	Hex code	Description
4100		MOV DPTR,#4400	90	Move data 4400 to DPTR
4101			44	
4102			00	
4103		MOVX A,@DPTR	E0	Move data from DPTR to Accumulator.
4104		MOV 0F0,A	F5	Move data from Accumulator to B register.
4105			F0	
4106		INC DPTR	A3	Increment DPTR content once.
4107		MOVX A,@DPTR	E0	Move data from DPTR to Accumulator.
4108		MUL AB	A4	Multiply the Accumulator content and B register.
4109		INC DPTR	A3	Increment DPTR content once.
410A		MOVX @DPTR,A	F0	Store the Accumulator content to DPTR.
410B		INC DPTR	A3	Increment DPTR content once.
410C		MOV A,0F0	E5	Move the data from B register to
410D			F0	Accumulator.
410E		MOVX @DPTR,A	F0	Store the Accumulator content to DPTR.
410F	Loop2	SJMP LOOP1	80	Jump to loop1.
4110			41	
4111			1C	

OUTPUT:**INPUT DATA:**

4400: 23

4401: 32

OUTPUT DATA:

4404: 6D

4405: 06

CONCLUSION:

Thus an assembly language program to multiply two data's was written and executed using 8051 micro controller kit.

Ex. No: 19**16 BIT DIVISION****AIM:**

To write an assembly language program to divide two 16 bit data's using 8051 Micro controller.

APPARATUS REQUIRED:

- 8051 Microcontroller kit.
- Power Supply.

ALGORITHM:

Step1: Start the program.

Step2: Load the two data's into Accumulator and B register.

Step3: Divide the two data's.

Step4: Store the result into the memory.

Step5: Stop the program.

MNEMONICS:

```

MOV      DPTR,#4400
MOVX     A,@DPTR
MOV      0F0,A
INC      DPTR
MOVX     A,@DPTR
DIV      AB
INC      DPTR
MOVX     @DPTR,A
INC      DPTR
MOV      A,0F0
MOVX     @DPTR,A
LOOP1:   SJMP    LOOP1

```


TABLE:

Memory	Label	Mnemonics	Hex code	Description
4100		MOV DPTR,#4400	90	Move data 4400 to DPTR
4101			44	
4102			00	
4103		MOVX A,@DPTR	E0	Move data from DPTR to Accumulator.
4104		MOV 0F0,A	F5	Move data from Accumulator to B register.
4105			F0	
4106		INC DPTR	A3	Increment DPTR content once.
4107		MOVX A,@DPTR	E0	Move data from DPTR to Accumulator.
4108		DIV AB	84	Divide the Accumulator content and B register.
4109		INC DPTR	A3	Increment DPTR content once.
410A		MOVX @DPTR,A	F0	Store the Accumulator content to DPTR.
410B		INC DPTR	A3	Increment DPTR content once.
410C		MOV A,0F0	E5	Move the data from B register to
410D			F0	Accumulator.
410E		MOVX @DPTR,A	F0	Store the Accumulator content to DPTR.
410F	Loop2	SJMP LOOP1	80	Jump to loop1.
4110			41	
4111			1C	

OUTPUT:**INPUT DATA:**

4400: ED

4401: 23

OUTPUT DATA:

4404: 06

4405: 1B

CONCLUSION:

Thus an assembly language program to divide two data's was written and executed using 8051 microcontroller kit.



INTERFACING PROGRAMS

E & I DEPT.

Ex. No: 20 STEPPER MOTOR INTERFACING USING 8085

AIM:

To write a program for interfacing stepper motor and to run the motor in different directions and in different speeds.

ALGORITHM:

Step1: Start the program.

Step2: Load HL registers pair with memory address at look up.

Step3: Move the contents of HL pair to accumulator.

Step4: Out the contents of accumulator to run the motor.

Step5: Decrease b register. If register content is not zero then rotate the motor continuously.

Step6: If zero then move to the Seining of the program.

Step7: Stop the process.

THEORY:

STEPPER MOTOR:

A motor in which the rotor is able to assume only discrete stationary angular position is a Stepper Motor. The rotary motion in a stepper motor is a stepwise manner from one equilibrium position to another.

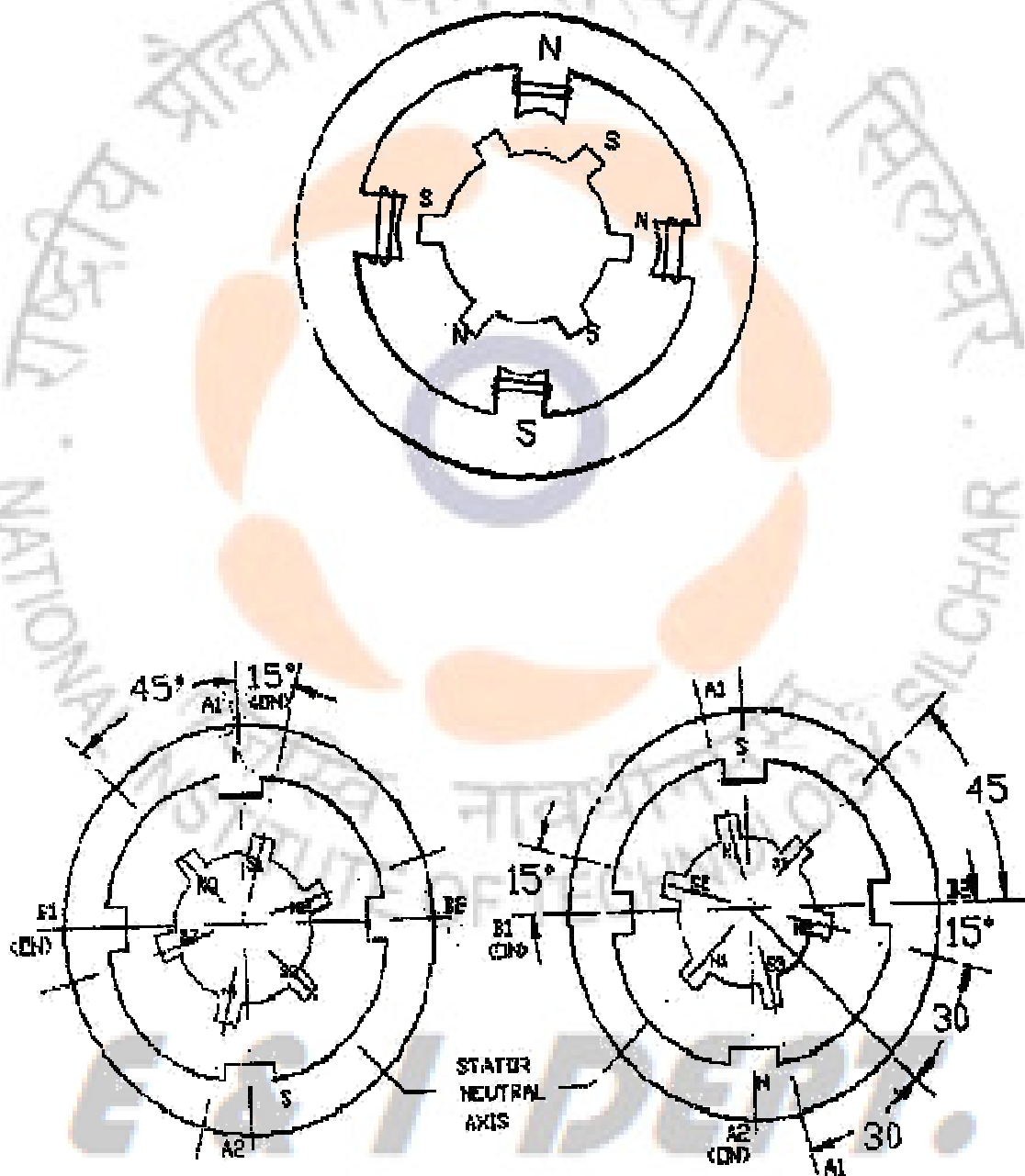
CONSTRUCTIONAL FEATURES:

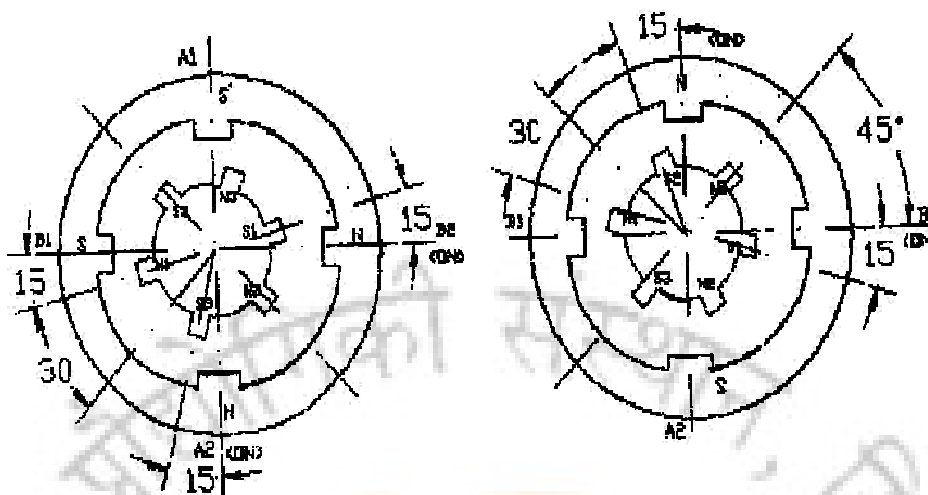
A stepper motor could be either of the reluctance type or of the permanent magnet type (PM). A PM stepper consists of multiphase stator and two part permanent magnet rotor. The VR stepper motor has unmagnified rotor. PM stepper motor is the most commonly used type. The basic two phase stepper motor consists of two pairs of stator poles. Each of the four poles has its own winding. The excitation of any winding generates a north pole (N), a south pole (S) gets induced at the diametrically opposite side.

As shown in the figure the four pole structure is continuous with the stator frame and the magnetic field passes through the cylindrical stator annular ring. The rotor magnetic system has two end faces. The left face is permanently magnetized as South Pole and their

right face as North Pole. The South Pole structure and the North Pole structure possess similar pole faces. The North Pole structure is twisted with respect to the South Pole structure by one pole pitch.

Stepper Motor Cross-sectional View





In an arrangement where four stator poles and three poles of rotor poles, there exists 12 possible positions in which a south pole of the rotor can lock with a north pole of the stator. From this it can be rotated that the step size is

$$= \frac{360^\circ}{N_s * N_r}$$

Where, N_s is the number of stator pole pairs N_r is the number of pairs rotor pole

Generally step size of the stepper motor depends upon N_r . These stable positions can be attained by simply energizing the winding on any one of the stator poles with a DC. There are three different schemes available for 'stepping' a stepper motor. They are,

- Wave Scheme
- 2-Phase scheme
- Half stepping or mixed scheme

2-PHASE SCHEME:

In this scheme any two adjacent stator windings are energized. There are two magnetic fields active in quadrature and none of the rotor pole faces can in direct alignment with the stator poles. A partial but symmetric alignment of the rotor poles is of course possible.

Typical equilibrium conditions of the rotor when the windings on two successive stator poles are excited are illustrated. In Step (a) A1 and B1 are energized. The pole-face S1 tries to align itself with the axis of A1 (N) and the pole-face S2 with B1 (N). The North Pole N3 of the rotor finds itself in neutral zone between A1 (N) and B1 (N). S1 and S2 of the rotor position

themselves symmetrically with respect to the two stator north pole.

Next when B1 and A2 are energized S2 tends to align with B1 (N) and S3 with A2 (N) of course. Again under equilibrium conditions only partial alignment is possible and N1 finds itself in the neutral region midway between B1 (N) and A2 (N) [Step (b)]. In Step (c), A2(N) and B2(N), respectively, with N2 in the neutral zone. Step (d) illustrates the case when A1 and B2 are ON.

The step angle is 30° as in the two phase's scheme. However the rotor is offset by 15° in the two phase's scheme with respect to the wave scheme. A total of 12 steps are required to move the rotor by 360° (mechanical) Two Phases drives produce more torque than the wave drives.

MNEMONICS:

```

START:  LXI  H, LOOK UP
        MVI  B, 04
REPT:   MOV  A, M
        OUT  0C0H
        LXI  D, 0303H
DELAY:  NOP
        DCX  D
        MOV  A, E
        ORA  D
        JNZ  DELAY
        INX  H
        DCR  B
        JNZ  REPT
        JMP  START
LOOK UP: DB  09 05 06 0A
  
```

TABLE:

LOOK UP TABLE

Step	Anticlockwise				Clockwise			
	A1	A2	B1	B2	A1	A2	B1	B2
1	1	0	0	1	1	0	1	0
2	0	1	0	1	0	1	1	0
3	0	1	1	0	0	1	0	1
4	1	0	1	0	1	0	0	1

TABLE:

Memory	Label	Mnemonics		HEX CODE	Description
		Instruction	Operand		
4100	START:	LXI	H, LOOK UP	21	Load HL pair with memory address at Look Up
4101				1A	
4102				41	
4103		MVI	B,04	06	Move immediate the given data to B register
4014				04	
4105	REPT:	MOV	A,M	7E	Move content of memory to Acc.
4106		MOV	[2000], AX	03	Out the content of Accumulator to C0 port address
4107				C0	
4108		LXI	D, 0303H	11	Load the data 0303H to D register
4109				03	
410A				03	
410B	DELAY:	NOP		00	Perform No operation
410C		DCX	D	1B	Decrement address of DE pair once
410D		MOV	A,E	7B	Move E register content to Acc.
410E		ORA	D	B2	Perform OR operation With Acc.
410F		JNZ	410B	C2	Jump on no zero to the
4110				0B	instruction at specified memory

4111				41	Address
4112		INX	H	23	Increment HL pair address once
4113		DCR	B	05	Decrement B register content once
4114		JNZ		C2	Jump on no zero to the
4115				05	instruction at specified memory
4116				41	Address
4117		JMP	START	C3	Jump to the instruction at
4118				00	specified memory
4119				41	
411A		LOOK	UP	09	Data will be stored in the location
				05	
				06	
				04	

CONCLUSION:

Thus the stepper motor is rotated by varying the speed using COUNT operation and its direction is also changed using program written and executed using 8085 micro processor kit.

Ex. No: 21 INTERFACING WITH PROGRAMMABLE INPUT OUTPUT – 8255

AIM:

To initialize port A as input port and port B as output port in mode 0. To input the data at port A and set by the spot switches and to output the same data to port B to glow the LEDs accordingly.

APPARATUS REQUIRED:

- 8085 Microprocessor Kit
- 8255 microprocessor programmable input/output
- Power Supply

PRODUCTION:

The 8255 has been displayed as general purpose programmable I/O device compatible with intel inputs. It contains three 8 bit parts which can be configured by software means to provide any one of the three programmable data transfer available with 8255.

PORT A

One 8 bit data output latch/ buffer and one 8 bit data input latch. Port A can function as input or output ports in three modes. The details are given in the following section.

PORT B

One 8 bit data output latch/ buffer and one 8 bit data input latch. Port B can function as input or output ports in two modes

PORT C

One 8 bit data output latch/ buffer and one 8 bit data input latch. Port C can function as simple input or output port. Thus port can be divided into two four bit ports which intern can function as sample input or output port. In addition the port C line can be used for the control signal outputs and the status signal outputs in conjunction with port A and port B.

GROUP A AND GROUP B CONTROLS:

The three ports of 8255 have been divided into two groups group A and group B. Group A contains port B and port C higher address lines. Group B contains port B and port C lower address lines.

The ports are configured as input are output by command window contains information such

as “mode”, “bit set” etc. In short command window decides. Whether the port is input port or output port and modes of transfer through a port.

Each of the control blocks accepts command to its associated ports.

CONFIGURATION 8255 WITH A MICROPROCESSOR:

The 8255 has all the necessary hardware for direct interfacing with 8116 bit bus. Data communication and configuration for direct data transfer can be done using the fal registers, namely three ports A, B and E and the control register available in the chip, register can be accessed with the help of A0 and A1 pin lines which are connected to the lower order bits A1 and A2 of the 8 bit microprocessor unit. The port registers are read / write register where as write registers is a control register.

PROGRAMMING THE 8255:

The control word can be programmed to configure the ports in a write variety of functional characteristics; the mode definition format is shown.

Port lines have been divided into two groups: Group A and Group B. Group A can be configured in these modes, mode 0, mode 1 and mode 2 where as group **B** can be configured into two modes mode 0 and mode 1.

The control word is 8 bit wide. Bit 7 decides whether the mode set of operation in bit set and reset operation is selected.

With Bit 7=1 Bit 6,5,4 and 3 to set the modes of group A while Bit 2,1 and 0 are to set the modes of group B. Detailed operation will be discussed later.

PROCEDURE:

Initialise the port A as input port and out to control read input from port A and out to the control port B. store the content of Accumulator in add 4500.

MNEMONICS:

```
ORG 4100H
MVI A, 90
OUT 0C6H
OUT C6H
IN COH
```

```
STA 4500H  
HLT
```

OBSERVATION:

Enter the program starting from the USER RAM address set a known data at the spot switches. Execute the program. The Above program initialises port A as an input port and port B as an out port. The data set by SPOT switches setting is input to the accumulator and is outputted to port B and the data output at the LEDs is the same, as that set by the SPOT switches settings. This input value from the accumulator is stored at 4500H.

CONCLUSION:

Thus a program to initialise port A as input port and port B as output port in mode 0 in processor 8255 was performed and their output was verified.